

Kanon M3: Pull Mode and the Reference Consumers

One verification, two modes — and what a read costs in each

v1.0

2026-10-04

Equilibrium Group Oy
Meritullinkatu 1
Helsinki
Finland

Logos
RFP-020

TABLE OF CONTENTS

Scope	4
1. What M3 delivered, in one page	5
2. The pull library	7
2.1 What it is, and how little it adds	7
2.2 Independence is a gate, not a test	7
2.3 The signer set is the consumer's, and cannot be grown by a payload	8
2.4 Typed-error parity	9
3. The two reference consumers	10
3.1 Asset-pair verification, and the obvious way to get it wrong	10
3.2 Which account to believe	11
3.3 Staleness, in both directions, with different causes	11
3.4 Refuse-on-unavailable, and why it is named for what breaks it	11
3.5 Why the roster lives in an account, in both consumers	12
3.6 On a real sequencer	12
4. What it costs, per mode	13
4.1 What a consumer pays for a usable price	13
What the figures deliberately exclude, and the correction that produced them	14
4.2 What a precompile would be worth	14
Build ECDSA first, and what that recommendation rests on	15
4.3 What a transaction costs, whole	16
This changes the headline rather than qualifying it	16
Where the reading goes	17
What is still outside	17
5. A program id that is a property of the source	18
5.1 What was found	18
5.2 What it would have cost	18
5.3 The change	19
5.4 The consequences, including the inconvenient ones	20
5.5 What this answers, and what it does not	20
6. Evidence	22
6.1 The suite	22
6.2 End to end	22
6.3 The gates	23
6.4 Requirement status	24
7. What M3 is not	25
7.1 Where a consumer gets the payload	25
7.2 The pins have not moved	25
7.3 Three claims that rest on something weaker than a measurement	25
7.4 Reproducibility, as far as it goes	26
7.5 Still M4's	26
8. Open with Logos	27
8.1 Answered by M3	27
8.2 Standing, and re-checked today	27
9. Reproducing this	30

Author: Equilibrium Group Oy

Status: Milestone deliverable

Created: 2026-10-04

Label: pull-mode

Client: Logos

Target: RFP-020 milestone M3, the ten tasks M3-01 to M3-10

Scope

M1 built the engine and M2 built the first vehicle. M3 builds the second, and in doing so turns ADR 2's "one verification, two modes" from a design intention into something the type system enforces: `pull-lib` is one function over `verifier-core`'s own types, returning `verifier-core`'s own error, with no aggregator type anywhere in its signature and no path that could reach one.

Around that function M3 adds the two reference consumer programs RFP-020 asks for — one per mode, holding the same limit order so a reader can put them side by side — and the cost work that answers what a consumer pays to obtain a usable price. That last question is where the two modes stop being variations on one design, and it is the first time this project has been able to answer it.

This report keeps M2's shape, because the milestone has the same character: sections 2 to 4 each state what a piece is for, what was built, and how it is proven, at the level of behaviour rather than code. Section 5 is new and has no counterpart in M1 or M2 — it is a defect found in review, what it would have cost in deployment, and the change that closed it. Section 7 draws the boundary, and section 8 states what remains open with Logos.

The organising idea is unchanged: nothing is claimed here without a test or a CI gate behind it. Every behaviour in sections 2 to 3 is asserted by the suite, every figure in sections 4 and 5 is generated by the code that asserts it, and the requirement mapping in section 6 is machine-checked, so a claim of completeness cannot quietly outrun its evidence.

Full detail is in the delivered repository: `TRACEABILITY.md` maps every RFP-020 requirement to its implementation and evidence, `adr/` records the forty-six design decisions, and `COSTS.md` is the cost report in full — which this milestone roughly doubles in size.

1. What M3 delivered, in one page

Pull mode is implemented, demonstrated in a program, and priced. `pull-lib` takes a payload and a consumer's own configuration — data service, feed, signer roster, threshold, window and expected asset pair — and returns a verified price or a typed error, with no dependency on the aggregator's price account and no transaction sent to anything. Two reference consumer programs show what each mode obliges a consumer to do, and the pull one has been exercised across a standalone sequencer.

All ten of M3's tasks are complete. M3-01 to M3-10, with nothing deferred and nothing moved out of the milestone.

area	what stands
Pull library	one function over <code>verifier-core</code> 's types, returning <code>verifier-core</code> 's error; independence from the aggregator asserted in CI rather than reviewed — section 2
Typed-error parity	the two modes answer a shared cause with the same value, asserted over six causes and structural for the rest; four asymmetries named rather than smoothed over — section 2
Reference consumers	one per mode, holding the same limit order, each showing asset-pair verification, staleness handling, typed-error handling and refuse-on-unavailable — section 3
Cost, per mode	what a consumer pays for a usable price in each mode, what a precompile would be worth to each, and what a whole transaction costs rather than a function body — section 4
Reproducible program ids	a guest's image id no longer depends on the directory it was built in; a release can now be independently verified by rebuilding it — section 5

The headline is the per-mode read, and it is a large number. A pull consumer pays **3,041,385 cycles** to obtain a usable price. A push consumer pays **6,803**. That is a factor of **447**, and 96% of it is signature recovery: the pull consumer has nothing published to fetch, so its read *is* a verification, and it pays that on every read. This is the figure that tells an integrator which mode to choose, and it has not existed before this milestone.

The second headline corrects the first, and both are published. A body is not a transaction. Running the product ELF's over the inputs LEZ would hand them puts a pull settlement at **3,525,809 cycles** against a push settlement's **324,667** — **ten times apart as transactions**, where their bodies are 227 times apart, because everything outside a body is reading and dispatch and neither scales with what the body does. A push settlement is 13,404 cycles of body inside that 324,667-cycle transaction: four percent of it. Both ratios answer real questions

and neither substitutes for the other: 447 is a read against a read, which is what a consumer author comparing implementations wants, and 10 is a transaction against a transaction, which is what somebody sizing a chain wants. Every transaction measured fits inside LEZ's 33,554,432-cycle budget, the largest at **10.51%** of it, which is what P1 asks for in both modes.

Requirement status moved accordingly. Five requirements went from *partial* or *planned* to **verified** — F9, U6, U7, SEC1 and SEC2 — taking the RFP-020 tally to **11 verified, 13 partial, 12 planned** of 36, against 6, 16 and 14 at M2.

Nine new architecture decision records take the total to forty-six. Seven questions are open with Logos (section 8), every one re-checked against its upstream repository on 2026-10-04 — and one of the seven, the reproducibility question M2 could not answer, **M3 answers affirmatively and by measurement** (section 5).

What M3 is not: the SDK, the relayer, the CLI and the operator tooling are M4's, and remain the placeholders M1 declared them. Section 7 draws that boundary.

2. The pull library

2.1 What it is, and how little it adds

F9 asks for a public-mode pull verification library: given a payload, a configured (`dataServiceId`, `feedId`, authorised signer set, M-of-N threshold, `maxAge`) and the consumer's expected (`base_asset`, `quote_asset`), return the verified price and timestamp or a typed error, with no dependency on the aggregator's price account.

`pull-lib` is one public function, `verify_price`, over `verifier-core`'s own types, returning `verifier-core`'s own `VerifyError`. The only type it adds is `PullConfig`: two public fields on top of a `FeedConfig`, so there is still one validated configuration shape behind both modes.

That thinness is the design, not an economy. ADR 2 promised “one verification, two modes”. A second crate that re-stated the verification contract would make that a claim two codebases have to keep agreeing on; a function that returns the first crate's error unchanged makes it an identity. `[M3-01:01]` records the one place the crate takes a position of its own rather than delegating — the clock is an account, not a parameter, and is refused unless it is the pinned every-block one — and records what that does *not* establish, which section 3 picks up.

`a_consumers_own_configuration_verifies_a_captured_payload` runs all five captured feeds through it with a consumer-supplied signer set, threshold and window, and no feed registered anywhere. It asserts that every captured signer was reported rather than that the threshold was met, because the second cannot fail: a verification below the threshold is an error, not a quiet result.

The `dataServiceId` the requirement names is carried, and `the_data_service_label_changes_nothing_about_a_verification` is what keeps that honest. The label cannot be checked against anything: a payload names no data service, and the one envelope slot that could carry one sits outside every signature. So the signer set is the binding and the label is the consumer's own record of where its roster came from. Stating that plainly is better than a check that looks like one and is not.

2.2 Independence is a gate, not a test

M3-02 is the half of F9 that says *no dependency on the aggregator's price account*, and it is enforced by a CI job rather than a test — on purpose.

No test can observe a dependency that is absent. The way this requirement would be lost is not a failing assertion; it is an addition that compiles, passes everything, and quietly couples the pull path to the push one. So `ci:pull-independence` pins the resolved build closure of `pull-lib`, of `reference-consumers/pull`, and of that consumer's own guest workspace to an **allow-list** — `verifier-core` and `kanon-clock`, nothing aggregator-side.

Four details in that job are load-bearing:

- **An allow-list, not a denylist.** A denylist only refuses the reuse somebody thought of.
- **Everything non-registry is listed**, reached by path or by git alike. The canonical price account is declared in `twap_oracle_core` and arrives by git, so a list of this repository's own crates would have left the aggregator's account type reachable. Registry packages are `cargo deny`'s business.
- **Every feature and every target**, not the runner's defaults. An optional dependency behind a feature nobody enables, and a dependency declared for the `riscv32` guest this crate actually ships into, are both ways past a default walk.
- **Dev-dependencies are outside it**, which is what leaves the parity test of the next section free to call both paths in one file.

The third walk is the one the requirement is really about, and [M3-06:01] records why: the artefact somebody copies is the *program*, not the library, and a program is a guest binary. `cargo tree` resolves a package rather than a binary, and the package holding the aggregator's guest depends on `aggregator-program`, so a consumer built there would sit in a closure containing the push aggregator with no walk able to say it did not use it. Giving the consumer its own guest workspace is what makes the closure an assertion about the program.

Both directions were run: injecting `aggregator-program` into the guest manifest brings `kanon-idl` and `twap_oracle_core` with it, and fails the diff.

2.3 The signer set is the consumer's, and cannot be grown by a payload

SEC2's pull half says the authorised set comes from the consumer and never from the payload. Verification does read signers out of the payload — recovering one from each package that carries the requested feed is the whole mechanism — but **a recovered address is only ever a lookup key**. It is accepted when it already appears in the separately supplied roster, and is otherwise the refusal; it never joins the roster, and never joins the set a later package is checked against.

`verify_price` takes the roster and the payload as separate arguments, and the roster as a shared borrow, so nothing in the call can grow it.

`a_roster_narrower_than_the_payload_refuses_rather_than_narrowing` states the operational reading: the set being the consumer's does not make it a dial.

A shared borrow proves immutability during a call and says nothing about where the roster came from. A consumer that copied one out of its own instruction data would satisfy every signature in `pull-lib` and defeat the requirement entirely. That boundary is a property of the program rather than the library, and section 3 is where it is discharged.

2.4 Typed-error parity

U6 asks for clear, actionable errors for every failure mode. M3-04 asserts that the two modes answer a shared cause with the **same value** — not with two taxonomies kept in step.

`every_shared_cause_is_the_same_value_in_both_modes` covers five: a malformed payload, an unauthorised signer, a threshold the payload cannot reach, a stale package, and one dated past the allowed skew. `an_asset_mismatch_is_the_same_refusal_from_two_carriers` covers the sixth separately, because the pair a caller claims is account state in push mode and an argument in pull mode.

Six are measured; the rest are structural, and the report says which. `InvalidSignature`, `ValueOutOfRange`, `ScalingOutOfRange`, `TimestampMismatch` and `TooManyPackages` need a crafted payload that only `verifier-core`'s `cfg(test)` builder produces (ADR 21). For those, the identity rests on `verify_feed` taking no argument naming its caller (ADR 23) and on `pull-lib` returning the error unchanged. That is a sound argument, and it is a different kind of evidence from a comparison — stating which is which is better than implying every mode was compared.

Four asymmetries are named rather than smoothed over, and two of them are asserted:

asymmetry	how it is handled
The clock	<code>SubmitError::Clock</code> in push, <code>VerifyError::NoClock</code> in pull — one leaf cause under two wrappers, routed through the same <code>clock_code</code> . <code>a_clock_failure_is_one_cause_under_two_wrappers</code> pins both leaf values, the difference in wrapper, and the number they share
The claimed pair	arrives by different carriers; asserted by <code>an_asset_mismatch_is_the_same_refusal_from_two_carriers</code>
Configuration failure	reaches push when it decodes a stored feed, and reached a pull consumer earlier, at <code>FeedConfig::try_new</code> . No test — described, not asserted
The account layer	push's alone. <code>every_submit_cause_is_classified</code> holds it, and its value is the wildcard-free match forcing the next <code>SubmitError</code> variant to be classified

That last gate earned its place this milestone: **the push taxonomy grew four times during M3**, and each time the match refused to compile until the new variant was classified.

3. The two reference consumers

U7 asks for two reference consumer programs, one per mode, each showing asset-pair verification, staleness handling, typed-error handling and refuse-on-unavailable. Both have landed.

They hold the same limit order on purpose. `open_order` records a price somebody will trade at; `settle` fills it if the market has reached that price. The domain is deliberately thin, so the two crates differ in exactly one thing — where a price comes from — and a reader can put them side by side. What a reader should take from them is the seam, not the trade.

consumer	where its price comes from
<code>reference-consumers/pull</code>	verifies a RedStone payload inline, against a roster the program itself governs
<code>reference-consumers/aggregator-read</code>	reads the price account the push mode wrote, and sends this adaptor no transaction at all

The ratio inside the pull consumer is worth naming, because it is the one the requirement actually implies: **two of its eight instructions are the limit order**. The other six are the authority and the per-feed trust accounts. The verification call is one line; everything around it is what makes the roster the consumer's rather than its caller's.

3.1 Asset-pair verification, and the obvious way to get it wrong

This is the property where satisfying the words does not satisfy the requirement, so it is worth stating in full.

Passing the trust account's own pair as `verify_feed`'s expected pair holds a value against itself. `AssetMismatch` becomes unreachable and nothing is verified. And the failure that leaves open is not exotic: nothing on the wire says which assets a feed prices, and no signer attests to it. A registration labelling RedStone's BTC feed as ETH/USD verifies real BTC packages, meets the threshold, passes every freshness check, and fills an order whose owner believed they were trading ether.

The owner's own expectation is the only thing that can catch it. So `open_order` takes the pair the owner is signing for, refuses the order if the registration disagrees, and stores it — `a_mislabelled_registration_cannot_fill_an_order_its_owner_did_not_intend`.

On the read side the same property has a different shape, because there are three records and two comparisons: the price account's pair is the aggregator's claim, the source account's is this consumer's authority's, and the order's is its owner's.

`a_price_account_pricing_another_pair_is_refused` and `an_order_cannot_be_filled_under_a_pair_its_owner_never_signed_for` are both live, and removing either fails only its own test.

3.2 Which account to believe

A reading consumer sends the aggregator no transaction, so what it has to decide is **which account to trust** — and nothing in the six published fields says who wrote one. `source_id` is the constant `RedStone` on every Kanon feed, so it names the data's origin rather than a program.

The only binding is the address, which hashes the aggregator's program id, which is its image id. `read_price` therefore derives that address from the feed's own registration and refuses any other account before it reads a field. The owner is checked as well as the address, because an address is not a guarantee about who filled it. Three tests, three states:

- `a_price_account_the_caller_chose_cannot_fill_an_order`
- `an_account_that_is_not_this_feeds_price_account_is_refused_first`
- `an_account_squatting_the_address_is_not_the_aggregators`

3.3 Staleness, in both directions, with different causes

Both consumers hold a window neither the payload nor its sender can widen, and both cover both edges with **distinct** errors — because an operator chasing a clock skew needs to be told which direction it is out.

	pull consumer	read consumer
far edge	<code>a_payload_past_the_registered_window_cannot_fill_an_order</code>	<code>a_payload_older_than_100_millisecond_past_the_window_is_stale</code>
near edge	<code>a_payload_dated_past_the_clock_cannot_fill_an_order</code>	<code>a_price_dated_further_ahead_than_skew_explains_is_refused</code>

The read consumer's window is its own rather than the aggregator's, because a published price starts ageing the moment it is written.

3.4 Refuse-on-unavailable, and why it is named for what breaks it

`no_refusal_returns_a_post_state` is the assertion in both crates, and it is named for the failure rather than the check. A consumer that answered a failed verification with `ok` and an unchanged account would leave a caller unable to tell *“the market has not reached your limit”* from *“nobody could verify a price”* — and a caller that cannot tell those apart retries the wrong one.

Six real causes are put through it in the pull consumer, each an error rather than a quiet no-op. On the read side the case the name is about is

`a_feed_nothing_has_been_published_for_cannot_fill_an_order`: a registered feed before its first

submission is a pristine account, and answering that with a zero price rather than an error is exactly the unsafe default U7 forbids.

Typed errors in both are a leaf-per-cause enum with **no wildcard arm** over `VerifyError`, so a variant added upstream stops the consumer compiling until somebody decides what to tell its callers. `no_two_causes_answer_with_the_same_number` counts every leaf across all blocks rather than per module, since a collision between blocks is the one a per-module test cannot see, and `the_number_a_caller_sees_is_the_frameworks_offset_one` pins the value a caller actually reads.

3.5 Why the roster lives in an account, in both consumers

The obvious way to give a consumer a signer set it owns is to compile one in. Nothing a caller sends can reach a constant, so it looks like the strongest possible answer. Both consumers reject it, for reasons that turn out to be the same reason.

On the pull side, RedStone rotates. ADR 30 is why the push path has `update_signer_set` rather than a table, and a compiled roster would meet that certain event with a redeployment — which in LEZ moves every derived address and abandons every open order. [M3-06:02] records the decision: an authority, a `FeedTrust` account per feed, and a rotation that is a transaction. A roster in an account the consumer itself owns is neither a dependency on the aggregator nor a registration against it, so it is not what F9 forbids, and SEC2's "comes from the consumer" is answered better by governed state than by a literal — because governed state also shows *how* a consumer governs its set.

On the read side, a compiled aggregator id looked more defensible: the address would simply go dead and every read would refuse, which is the safe failure U7 asks for. [M3-05:01] records why it still loses. It fails safely and **recovers badly** — following an aggregator rebuild means rebuilding the consumer, which moves the consumer's own accounts and abandons every open order. An aggregator fix would cost the order book. The trigger differs, the mechanism is the one ADR 33 describes, and the answer is the same account-behind-a-gate on both sides. `an_order_survives_the_aggregator_being_rebuilt` is what makes that arrangement more than plumbing.

3.6 On a real sequencer

M3-07 runs the pull consumer against a standalone LEZ sequencer. It establishes an authority, registers a roster, opens an order, and settles it against a payload it verifies itself — all on chain, with no aggregator deployed and no price account read.

The second half of that test is what a compiled roster could not have done: **the authority rotates to a disjoint signer set, and an order opened afterwards settles against a payload only the new signers signed**. That is [M3-06:02]'s decision working rather than described.

4. What it costs, per mode

Three tasks, three questions, and they are reported in the order they correct one another: what a read costs (M3-08), what a precompile would be worth (M3-09), and what a whole transaction costs rather than a function body (M3-10).

Every figure below is generated and asserted by `methods/tests/read_cost.rs` and `methods/tests/bytes.rs`. None is transcribed: the same code that prints the table is what CI compares against, so a published figure and an asserted figure cannot drift apart.

4.1 What a consumer pays for a usable price

The M2 report priced a *submission*. This prices a **read** — what a consumer spends to obtain a price it can act on — and it is where the two modes stop being variations on one design.

	pull	push
the mode's read	3,041,385	6,803
settling, beyond the read	7,228	6,601
the whole <code>settle</code> body	3,048,613	13,404
<i>the read, against the 33,554,432-cycle budget</i>	9.06%	0.0203%
<i>the <code>settle</code> body, against the same budget</i>	9.09%	0.0399%

A pull read costs 447 times a push read, and that is not a tuning difference. The pull consumer has nothing published to fetch, so its read *is* a verification, and it pays that on every read. The push consumer reads six fields the aggregator already verified once, and recovers no signatures.

96% of the gap is signature recovery. Five-signer recovery measures 2,922,890 against a difference of 3,034,582, and `the_gap_between_the_modes_is_signature_recovery` asserts that share rather than leaving it as a claim. The other 4% is the rest of verification — the package walk, the hashing, the membership lookups — so the claim is that the gap is one component, not that it is only one component.

Two things in that table are worth reading against intuition:

The largest item in a push read is hashing, not decoding. `read_price` opens by checking that the account it was handed is the one this feed publishes to, and computing that address means `compute_pda` twice — once for the feed account, once for the price account derived from it — each a SHA-256 over its seeds. At 3,656 cycles that is 54% of the read, more than twice the

account's decode, and it is why the consumer's guest manifest pins `sha2` to the risc0 accelerator. **What push mode avoids is signature recovery, not cryptography.**

The pull side's verification agrees with M1's component table to within 722 cycles — 3,040,512 against 3,039,790 for the same five-signer payload. That agreement is the point: it means a pull read is an update's verification, and not something adjacent to it. The 722 is one clock decode and the library call around it.

What the figures deliberately exclude, and the correction that produced them

Everything a consumer holds *before* it reads — the registration's decode on both sides, and the order's on the pull side — is done in setup that cancels between the two stages being differenced.

An earlier draft did those decodes inside the read, and so charged the reference consumer's limit order to pull mode and a fifth of the push figure to push mode, which made the two residuals look four times further apart than they are. [M3-08:01] records the correction. It is reported here rather than quietly fixed because the corrected table is the one a reader will quote.

4.2 What a precompile would be worth

P3 asks for the delta between the in-program path and a hypothetical native ECDSA and keccak256 precompile, per mode. **LEZ has no such primitive**, so the answer rests on three things about it nobody can measure: what a call into it costs, what it hands back, and what it validates.

M3-09's decision is to publish **a function rather than a figure**. The crossing cost is carried as `c` and every number is given at both ends of M0's 1,000-to-10,000 band and at zero. Zero is not achievable and is not meant to be — it is the bound that holds whatever a real precompile costs, which is the useful thing to hand somebody deciding whether to build one.

A five-package verification does five hashes and five recoveries, so a precompile **removes 3,010,270 cycles and adds ten crossings, wherever that verification happens**. The saving is identical in both modes. What differs is how often a consumer pays it:

a read	pull	push
today	3,041,385	6,803
with a precompile, $c = 0$	31,115 (97.7×)	6,803 (1×
with a precompile, $c = 1,000$	41,115 (74.0×)	6,803 (1×
with a precompile, $c = 10,000$	131,115 (23.2×)	6,803 (1×

A pull consumer pays the whole saving on every read. A push consumer gets nothing — not a small amount, nothing, because a push read performs no recovery at all. Push mode's saving is real but it lands on the aggregator's update, which goes from 3,048,414 to between 38,144 and 138,144.

So for a feed updated once and read R times before its next update, a precompile is worth $3,010,270 - 10c$ per read in pull mode and the same figure divided by R in push mode. **It is worth R times more to a pull consumer.** That is P3's per-mode answer: one number, two frequencies.

Build ECDSA first, and what that recommendation rests on

Of the 3,010,270 a precompile removes, the recovery row is **97.10%** and the keccak256 row **2.90%**, at one, three and five signers alike.

Rows are not primitives, and the correction matters. The recovery row is everything `recover_signer` does: parsing the 65-byte signature, refusing a malleable one, the recovery, encoding the point — and then a *second* keccak256, because `address_of` hashes the 64-byte point to get an Ethereum address. On an estimate that takes that hash at the measured cost of the 77-byte message hash, the split by primitive is roughly **5.8% keccak256 against 94.2% ECDSA**. Nothing isolates the address hash, so **that one figure is an estimate and is the only unmeasured number in the section.**

The recommendation survives the estimate — ECDSA is the larger half whichever way the address hash is counted. It does **not** survive an arbitrary package shape: hashing grows with the signable span and recovery does not, so a feed publishing many data points per package would move the balance, and nothing here establishes where. Read “build ECDSA first” as scoped to packages the shape RedStone's primary feeds publish today.

A further point against the hashing side: most of that saving is already available without LEZ changing anything. This build declines risc0's keccak coprocessor, which would divide that row by about seven (ADR 6). **A keccak256 precompile would be competing with a configuration change.**

4.3 What a transaction costs, whole

Every figure above — and every figure in M1's and M2's reports — is a program **body**. A body is a fraction of a transaction. LEZ reads the inputs before any body is entered, then the dispatcher decodes the instruction, the generated validator checks the accounts, the body runs, and `Spe1Output` wraps the result. None of that is reachable by calling a function.

It is reachable by **running the program**, which is what M3-10 does: the product ELF's over the inputs LEZ would hand them, with the accounts at their real derived addresses.

	to be read	body	the rest	transaction	of budget
pull settlement	188,072	3,048,613	289,124	3,525,809	10.51%
push update	175,826	3,048,414	259,469	3,483,709	10.38%
push read	122,783	13,404	188,480	324,667	0.97%

This is P1's answer in both modes, and it is comfortable rather than tight. The largest transaction measured is 10.51% of the 33,554,432-cycle budget, and `every_transaction_fits_the_budget` asserts the limit rather than restating it.

"The rest" is taken as a difference, so it carries whatever the three harnesses disagree by as well — this project has measured two of them differing by 42 and 722 cycles on identical verification. At 188,480 to 289,124 that variance is a rounding error inside it, but the column remains a difference between measurements rather than a measurement of its own, and is reported as such.

This changes the headline rather than qualifying it

A pull settlement's body is **227 times** a push settlement's. As executed transactions the two are **ten times apart**. A push settlement is 13,404 cycles of body inside a 324,667-cycle transaction — **four percent of it** — because everything else is being read and being dispatched, and neither scales with what the body does.

Both ratios are published because they answer different questions. **What a mode's code costs** is the read table, and it is what a consumer author comparing implementations wants. **What a transaction costs** is this, and it is what somebody sizing a chain wants. Quoting the first as though it were the second is an error this project made in an earlier draft of the precompile section, which put the saving at 14× to 25× by adding an estimate of the instruction read to the body; the estimate was low and omitted everything but the instruction. Measured instead, **a precompile is worth about 6× to 7× against a transaction, against 22× to 80× against a body.**

Where the reading goes

The read half is measured separately, because it is the half a program can bound and the rest is the framework's.

a read of four empty inputs	4,426
an instruction word	113
a byte of an account's data	113
an account, on top of its data	16,156

The 113 is exact and linear to the cycle from 1,024 words to 16,384. It confirms ADR 26's "about 113" and **settles its unit: it is a word**, so multiplying a serialised size in bytes counts each word four times.

The last two rows are why the transaction figures are measured rather than assembled. Four framed reads cost something with nothing in them, and an account costs far more than its width, because deserialising an `AccountWithMetadata` builds a `Data`, an `AccountId` and a `ProgramId` rather than copying a span. **A word count times a rate understates every mode, and the push read by nearly half.**

What is still outside

Proving. Everything above is execution, which is what LEZ's per-transaction budget is denominated in.

5. A program id that is a property of the source

This section has no counterpart in M1 or M2. It is a defect found while reviewing a cost measurement, what it would have cost in deployment, and the change that closed it. It also answers a question M2 put to Logos and could not answer itself.

5.1 What was found

In LEZ, a SPEL program's id **is** its RISC Zero image id, and every account the program owns is a PDA derived from it. [M3-05:01] records what a moved id costs: the old accounts go dead, every open order is abandoned, and there is no migration path.

M3-08 reviewed a rule about not moving one by accident, and the review turned up something the rule could not survive:

A guest's image id depended on the absolute path the repository was checked out at. Same commit, one machine, one toolchain, --locked: two checkouts at two directories produced three different product ids.

The mechanism is ordinary once seen. Cargo derives -C metadata from a hash of the package id, and a package id's source id is location-independent for registry and git sources but **is the absolute manifest path** for a path dependency. That hash becomes rustc's StableCrateId, which appears in every v0 mangled symbol name, which changes the ELF, which changes the image id.

Diffing two such ELF's shows it exactly: the only differences are 178 symbol names whose crate disambiguators moved, for aggregator, aggregator_program, kanon_clock and verifier_core — our four path crates — and for nothing else. Every registry crate was stable, and so were the git dependencies. **The path never appears in the binary, only its hash, which is why grepping for it finds nothing.**

5.2 What it would have cost

Nothing shipped was wrong. CI always builds at one runner path, so it reproduces itself, and every deployed id was the id CI computed.

What was broken is **independent verification**. Anybody rebuilding a commit to check a deployed program's id would compute a different id, and therefore different account addresses, unless they happened to reproduce our directory. That is the property the deployment story had been assuming, and it did not hold — and it failed across two directories on one machine, which is a far weaker condition than the two-machine case M2's open question proposed testing.

5.3 The change

`methods/build.rs` now builds all three guest packages through `risc0-build`'s container path, which copies the tree to a fixed `/src` and builds there, so manifest paths — and with them the metadata hashes — stop depending on the host. **Verified rather than inferred: two checkouts at two paths, built this way, produce all three product ids byte-identical.**

Three details decide whether that holds:

- **The pin is by digest, and that is the load-bearing half.** `risc0-build` defaults to image `r0.1.88.0`; taking the default would swap the guest compiler under every figure in `COSTS.md`, all measured on guest `rustc 1.97.0`, and turn a change about *where* a guest compiles into a change about *what* compiles it. The image carrying that compiler is pinned as `r0.1.97.0@sha256:7ae0a27f...`, because **a tag is a mutable pointer, not a pin**: two builds of this commit far enough apart could resolve one to different contents and produce different ids — the same failure arriving slowly instead of immediately. All 31 cost assertions pass unchanged on that image.
- **A pin `risc0-build` will ignore is not a pin either.** `RISC0_DOCKER_CONTAINER_TAG` is consulted *ahead* of the configured image, so the build script refuses to run when it is set to anything else, rather than quietly building a different program and publishing its ids as these.
- **The guest list is derived, not restated.** `risc0-build` applies host options to any guest absent from the map it is handed, so a hand-kept list would be a second copy whose disagreement is silent — a fourth guest added to the metadata and not to the list would compile outside the container and have a directory in its id again, with nothing to say so. The build script reads the metadata instead.

Two rejected alternatives are worth recording, because both look right:

`--remap-path-prefix` cannot work. It is a `rustc` flag that rewrites paths in diagnostics and debug info, and `-C metadata` is computed by cargo **before** `rustc` is invoked.

A bind mount at a canonical path fixes the metadata hash and stops there. The guest ELF also embeds 19 `$CARGO_HOME` paths — panic locations from `k256`, `risc0-zkvm`, `spel-framework`, `tiny-keccak` and others — so an id built that way is still a function of whose home directory built it. Both experiments that found the path dependence ran on one machine with one `$CARGO_HOME`, which is why that second cause stayed invisible. **A container normalises it too, because it carries its own `/root/.cargo`.**

5.4 The consequences, including the inconvenient ones

The three product ids change once, here, and are now what a container at that digest produces from this source — reproducible by anybody who pulls it. They are recorded in [M3-08:02].

Docker with buildx becomes a requirement for building this repository, as it already was for the sequencer harness (ADR 34). The plugin is not incidental: `risc0-build` runs `docker build --output`, which only BuildKit provides, and the legacy builder's refusal names the flag rather than the cause. CI cannot catch that — GitHub's runners ship buildx — so the requirement is invisible exactly where it is exercised. `build.rs` checks for it first and fails naming it.

KANON_GUEST_BUILD=host remains, because the builder image is x86_64 linux and a developer on another architecture would otherwise be unable to build at all. What a host build must not be used for is anything published, and `the_guests_were_built_in_a_container` is what says so — a build warning is lost in a CI log and easier to lose locally. Cycle figures are unaffected either way: they are a property of the instructions, and the difference between the routes is confined to symbol names.

A clean containerised build is slower, measured at 5m 42s including the image pull and 3m 53s after. The runner cache covers `target/`, so the cost falls on cold caches rather than every run.

It does not fit on a stock runner, and the failure is not legible. The builder image is 1.73 GB compressed and the container resolves each guest workspace's dependencies inside itself. The first CI run failed twice on that one cause and only one of them said so: no space left on device in one job, and `ld` terminated with signal 7 [Bus error] in another — what a truncated mmap looks like when the disk fills under a linker. Every job that builds `kanon-methods` now reclaims the runner's preinstalled SDKs first.

A container inherits no environment, and the guests read some. Each program takes its genesis authority through `option_env!`, which resolves against the compiler's environment — now inside a container that inherits nothing. Left alone, every guest would have built without one, compiled cleanly, and refused every transaction at execution time with `this build configured no genesis authority`, which is how CI found it. The build script forwards what the guests read, discovered by scanning them for the call rather than by keeping a list.

5.5 What this answers, and what it does not

M2's seventh open question asked whether two machines at the same pins produce the same guest ELF, and recorded that nothing in the repository established it.

It is now answered affirmatively, and by measurement. A review of this change built the same commit on a second machine and obtained all three product ids identical to the ones recorded here. So a release can be independently verified by rebuilding it, which is the property the deployment story had been assuming without evidence.

Both halves are measured: the answer was no before this change and yes after it. What it does not extend to is a third architecture, since the builder image is x86_64.

6. Evidence

6.1 The suite

594 tests, all green, across thirty-four test targets in the product workspace, plus property tests that generate their inputs rather than choose them. The figure is the sum reported by `cargo test --workspace --locked`; section 9 gives the command. M2 delivered 360.

Where the growth went:

crate	tests
verifier-core	176
aggregator-program	142
reference-consumers/aggregator-read	99
reference-consumers/pull	90
methods (the cost suites)	52
kanon-clock	14
traceability	12
pull-lib	9

The two reference consumers account for 189 of them, which is the ratio section 3 describes: a thin limit order surrounded by the authority, the per-feed trust accounts and the refusals, because that surround is what the requirement is actually about.

`pull-lib`'s nine look thin beside the rest, and they are, deliberately. The library delegates almost everything to `verifier-core`, whose 176 tests cover the verification both modes run; duplicating them behind the second caller would assert that a function call forwards its arguments. What `pull.rs` does assert is the part that is this crate's — the consumer-supplied configuration path, the unauthorised-package refusal, and the re-exports a consumer needs to build a roster at all.

6.2 End to end

Four end-to-end tests run against a standalone LEZ sequencer, booted in CI from the same LEZ revision the product lockfiles resolve. These live in their own workspace, for the reason ADR [M2-19:01] records, and so are counted separately from the 594 above.

test	what it converts from a claim into an executed transaction
<code>the_sequencer_is_there_and_producing_blocks</code>	the harness itself
<code>the_push_path_verifies_and_publishes_across_a_real_sequencer</code>	M2's push submission
<code>the_five_feeds_register_on_the_public_testnet</code>	M2's feed registration, on a public network
<code>the_pull_path_verifies_and_settles_across_a_real_sequencer</code>	M3's : an authority established, a roster registered, an order opened and settled from a payload the consumer verifies itself — then a rotation to a disjoint signer set, and an order settled under it

The pull test deploys no aggregator and reads no price account. That shows the pull path does not **need** what section 2's closure walk proves it cannot reach — but `e2e` is outside that walk, and `aggregator-program` is a dev-dependency there, so its silence is a convention held by the file's own header rather than a gate. The gate is `ci:pull-independence`, and the distinction is worth keeping: a reader who takes the convention for the gate would look to the wrong place for a guarantee.

6.3 The gates

job	what it holds
<code>lint</code>	<code>fmt --check</code> and <code>clippy -D warnings</code> , now across the guest workspaces too, whose ELF's are program ids
<code>build-test</code>	the workspace builds and the suite passes, locked
<code>no-std</code>	<code>verifier-core</code> , <code>pull-lib</code> and <code>kanon-clock</code> still build for a bare-metal target with no <code>std</code> at all, which is the other half of <code>pull-lib</code> being usable inline
<code>pull-independence</code>	new. The resolved build closures of <code>pull-lib</code> , the pull consumer and its guest workspace match an allow-list, over every feature and every target
<code>guest</code>	the guest workspaces build as program ids depend on
<code>licenses</code>	the dual-licence allowlist, with three clarifications rather than a widened allowlist
<code>traceability</code>	the requirement matrix matches its sources
<code>sequencer</code>	the end-to-end suite against a standalone LEZ sequencer

Two guardrail jobs sit beside them: `guardrails` pins the M0 measurement workspace, and `product-costs` regenerates and asserts every figure in section 4.

6.4 Requirement status

requirement	at M2	at M3	what closed it
F9	planned	verified	the library, the closure gate, the consumer and the sequencer run
U6	partial	verified	typed-error parity, asserted over six causes and structural for the rest
U7	planned	verified	both reference consumers, all four properties each
SEC1	partial	verified	the unauthorised-signer refusal now covered in both modes
SEC2	partial	verified	the pull half: a roster the caller cannot reach, governed by the consumer's own authority

Across the whole RFP-020 inventory that is **11 verified, 13 partial, 12 planned** of 36 requirements, against 6, 16 and 14 at M2 and 1, 13 and 22 at M1.

The matrix is machine-checked rather than maintained: `cargo test -p traceability` fails if `TRACEABILITY.md` and its sources disagree, if any evidence names a test, CI job or document that does not exist, or if a task id written anywhere in the repository appears in no row. **That check is the deliverable, not the table** — an unverified matrix drifts into fiction by about the third milestone, which is this one.

7. What M3 is not

All ten of M3's tasks are complete, so unlike M2 this section records no shortfall. What it records is the edge of what the milestone establishes, stated here rather than left for a reader to find.

7.1 Where a consumer gets the payload

F9 is the library and the program that verify **bytes already in hand**. Where a pull consumer obtains those bytes is deliberately outside this milestone: the relayer's gateway fetch is M4-02 under F8, and what a consumer integrating pull should be told about fetching for itself belongs to S4's README and S5's doc packet, both still planned.

One thing that documentation owes a reader is worth naming now, because it is not obvious and it is not written down anywhere else. A pull consumer that fetches from RedStone's gateway at read time exposes the fetcher's address to RedStone. On a privacy-first chain that is a consideration rather than a footnote, and it is the reason the alternatives — taking the bytes from a relayer, or from the transaction itself — are worth spelling out beside the pattern. `reference-consumers/pull/src/lib.rs` names the trade where a consumer author will meet it; the recommendation is S5's.

7.2 The pins have not moved

M3 is delivered on **SPEL v0.6.0** and **LEZ v0.2.0**, the same pins M2 shipped on, and we have not changed them underneath it. Section 8 carries the open question about which pin the estate wants, and why moving it is a decision rather than an upgrade: it moves every program's image id, and therefore every derived account address.

The guest builder image pinned in section 5 is a second pin that has to move with ADR 8's, because it carries the guest compiler every figure in section 4 was measured on. A toolchain bump is now two edits, and a mismatch between them is a re-measurement nobody asked for.

7.3 Three claims that rest on something weaker than a measurement

Stated plainly so the scope of section 4 is not read wider than it is.

The precompile split by primitive is an estimate. Nothing isolates the `address_of` hash inside the recovery row, so the 94.2%-to-5.8% split rests on taking that hash at the measured cost of the 77-byte message hash. The row split — 97.10% to 2.90% — is measured; the primitive split is not. The recommendation survives either, which is why it is made.

“Build ECDSA first” is scoped to a package shape. Hashing grows with the signable span and recovery does not, so a feed publishing many data points per package would move the balance. Every captured package carries exactly one data point and a 77-byte span, which is what RedStone’s primary feeds publish today, and nothing here establishes where the crossover is.

Five of the eleven typed-error causes are structural rather than compared. Section 2 names them. They need a crafted payload only `verifier-core`’s `cfg(test)` builder produces, so the identity rests on `verify_feed` taking no argument naming its caller and on `pull-lib` returning the error unchanged.

One asymmetry has no test at all: the moment a configuration failure arrives differs between the modes — push meets it when it decodes a stored feed, a pull consumer met it earlier, at `FeedConfig::try_new`. That is described in the matrix and asserted nowhere.

7.4 Reproducibility, as far as it goes

Section 5 establishes that two machines at these pins produce identical product ids. **It does not extend to a third architecture**, because the builder image is `x86_64 linux`. The same limit applies to the published sequencer image, and a developer on another architecture has both escape hatches available — `KANON_GUEST_BUILD=host` and `KANON_SEQUENCER_RUNTIME=host` — with the understanding that ids from a host build are their own and must not be published.

7.5 Still M4’s

The SDK, the relay daemon, the CLI, the operator documentation and the monitoring remain the placeholders M1 declared them. Nothing in M3 advances them, and section 6’s requirement table shows thirteen requirements still partial and twelve still planned for that reason.

8. Open with Logos

Seven questions are open. **Every one was re-checked against its upstream repository on 2026-10-04**, immediately before this report was sent, and the findings below are that check rather than a restatement of M2's.

This re-check is a standing discipline rather than a courtesy. M1 v1.0 shipped asking Logos about a pull request that had merged a month earlier. **An ask upstream has already answered is worse than no ask.**

One of the seven is answered this milestone, and we answered it ourselves.

8.1 Answered by M3

7. Is a guest ELF bit-identical across machines at the pinned toolchain? — answered, affirmatively, by measurement.

M2 recorded that nothing in the repository established it. It is now established, and the honest version of the answer is that **it was no before this milestone and is yes after it**: a guest's image id depended on the absolute path the repository was checked out at, which two directories on one machine were enough to show — a weaker condition than the two machines the question proposed. [M3-08:02] records the mechanism and the fix, section 5 of this report states it in full, and a review of the change built the same commit on a second machine and obtained all three product ids identical.

What this does not extend to is a third architecture, since the builder image is x86_64. No action is needed from Logos; it is listed here because the question was asked of you and the answer is owed back.

8.2 Standing, and re-checked today

1. Which LEZ pin should the delivered adaptor carry, and when should it move? Re-checked 2026-10-04: SPEL's latest release is still **v0.7.0** (2026-09-17), with v0.7.0-rc.1 and rc.2 tagged after it as prereleases; LEZ's latest is still **v0.3.0** (2026-09-30). Nothing has moved in four days.

M3 is delivered on SPEL v0.6.0 and LEZ v0.2.0, unchanged. ADR 31 records that a SPEL bump and a LEZ bump are one decision and not two, because cargo unifies the two spellings of the LEZ crates only when source and tag match exactly. On later milestone branches the move has already been made — the M4 work resolves LEZ v0.2.4 and the M5-00 branch resolves v0.3.0 — so the migration is neither speculative nor unmeasured. **What we need is which pin the delivered adaptor should carry and when it should move, rather than a**

choice we make branch by branch. It is not a free upgrade: it moves every program's image id and therefore every derived account address, including the five recorded in `TESTNET.md`.

2. Where is the RFP-001 admin-authority extension going to live? `logos-co/spel` PR 212 is **still open, still a draft, and has had no activity since 2026-05-20** — four and a half months as of this check. The contingency shim ADR 14 describes is the live path, and the interface 212 offers turned out not to be the like-for-like substitution that ADR assumed. We need to know whether 212 is the intended home, whether the extension crate in a personal namespace is, or whether something else is, so the admin gating can move onto it rather than continue to diverge.

3. Three Logos crates ship without licence metadata. Re-checked 2026-10-04: `spel-framework-core` and `spel-framework-macros` declare no `license` field at v0.7.0, and `twap_oracle_core` on `lez-programs` main declares none either. Still missing in all three. They are not unlicensed — `lez-programs` ships a MIT LICENSE and `spel` ships both MIT and Apache-2.0 — but cargo-deny cannot see that. `deny.toml` carries three `[[licenses.clarify]]` entries rather than a widened allowlist, because a clarification names one crate and can be checked. **This remains the cheapest ask on the list:** a one-line `license = "..."` in each manifest removes all three entries and spares every downstream consumer making the same private judgement.

4. May a program claim a non-default, default-owned account? One rule pair, met twice. It strands a signer a program declines to claim, and it lets anyone put one unit of balance on a publicly derivable PDA and make that address permanently unwritable — including the admin config account, whose address takes no input an attacker cannot predict. ADR [M2-06:01] and ADR 33 record the consequences we designed around. **M3 shows the rule pair is not the aggregator's problem alone.** Both reference consumers derive their own accounts the same way and both had to carry the same refusal:

`a_squatted_config_address_is_refused_as_unusable_and_not_as_established` in the pull consumer's authority, and `an_account_squatting_the_address_is_not_the_aggregators` in the read consumer, with the registration path covered separately. Every program that owns a PDA on LEZ writes this check, or inherits the defect.

5. Can the canonical price account be made cheaper to depend on? Re-checked 2026-10-04: `twap_oracle_core` still pins `risc0-zkvm` = "3.0.5" exactly and still carries `uniswap_v3_math` to declare six Borsh fields, and the workspace root pins the same version. This is closer to a defect report than a preference: relaxing the pin, or splitting out a standalone account-type crate, would make the canonical account usable by the external adaptors it was written for. Kanon is one of those adaptors — and section 2 shows the cost concretely, since `twap_oracle_core` reaching this repository through `kanon-idl` is precisely what the pull consumer's closure walk has to reason about.

6. What happens to a price account when the program that created it is upgraded?

Recorded in ADR 22 and still open. Under LEZ's ownership model a program id is the hash of its binary, an account is bound for good to the id that created it, and no operation hands an account to a new binary. So an upgraded aggregator cannot update the price accounts the old one created, and consumers would have to follow it to new addresses.

M2 said we wanted the shape settled before M3 poured concrete around one of them. M3 has now poured it, and we are telling you which way. Both reference consumers answer it the same way and for the same reason — [M3-05:01] and [M3-06:02] — which is that the alternative fails safely and **recovers badly**: a compiled aggregator id makes the address go dead and every read refuse, which is correct, but following the rebuild means rebuilding the consumer, which moves the consumer's own accounts and abandons every open order. So what a consumer trusts lives in an account behind an authority gate, and a rebuild is a transaction rather than a redeployment. `an_order_survives_the_aggregator_being_rebuilt` is the assertion.

That is a decision about **our** reference consumers. It is not a decision about the estate, and the question it was an answer to is still yours: whether consumers should discover the account rather than pin it, or whether a small permanent program should own the accounts with the upgradeable one writing through it — the only arrangement that keeps the address fixed. Nothing LEZ documents answers it. We would rather align the reference consumers with whatever the estate chooses than have them become the de facto answer by being the only one written down.

9. Reproducing this

From the repository root, inside `nix-shell`:

```
cargo test --workspace --locked           # the 594 product tests, minutes
cargo test --release --locked -p kanon-methods # sections 4 and 5, built in the guest
cargo test -p traceability                 # the requirement matrix
cargo run -p traceability --bin render-traceability # regenerate TRACEABILITY.md
python3 scripts/capture-signer-sets.py --minutes 5 # re-capture the signer rosters
```

Docker with the buildx plugin is now required, for the reason section 5 gives: the guests are compiled inside a pinned container, and that is what makes a program id a property of the source rather than of the directory it was built in. `methods/build.rs` checks for buildx before anything else and fails naming it, because `risc0-build` runs `docker build --output` and the legacy builder's refusal names the flag rather than the cause.

`KANON_GUEST_BUILD=host` builds outside the container, for a developer on an architecture the `x86_64` builder image does not serve. **Cycle figures are unaffected** — they are a property of the instructions, and the difference between the routes is confined to symbol names — so everything in section 4 reproduces either way. **Program ids are not**: ids from a host build depend on the checkout path and must not be published, which `the_guests_were_built_in_a_container` is what asserts rather than warns.

The end-to-end tests resolve in their own workspace (ADR [M2-19:01]) and need a sequencer; `scripts/lez-sequencer.sh` fetches, starts and smoke-tests one, which is what CI does.

Everything section 6.3 describes runs on every push: `ci.yml` carries the lint, build-test, `no_std`, pull-independence, guest, licence, traceability and sequencer jobs; `guardrails.yml` re-asserts MO's frozen figures and the product cost tables; `lgs-build.yml` proves the toolchain path; `lez-sequencer-image.yml` publishes the prebuilt sequencer image CI boots.

document	contents
TRACEABILITY.md	every RFP-020 requirement, its status, its tasks and its evidence — the generated, machine-checked map of what is done and what remains
COSTS.md	the full cost report, its method, and how to regenerate it — roughly twice the size it was at M2, with the per-mode read, the precompile delta and the whole-transaction sections this milestone adds
MANIPULATION-ANALYSIS.md	the SEC3 analysis, the <code>maxAge</code> floor and ceiling, and what the analysis does not cover
FEEDS.md	the five feeds, their recovered signer rosters, publish cadence, and how to re-capture all of it
TESTNET.md	what is deployed on the public testnet, where, and how to check it without trusting the document
adr/README.md	all forty-six decision records, with a reading order and the open questions collected
m0/versions.md	every pinned version, why each is pinned that way, and the procedure for changing one

A cost-guardrail failure is a prompt to re-measure and re-publish deliberately, not necessarily a defect — but a published figure never moves without the change that owns it.